

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA1405

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

SCENARIO BASED TASK

Code of Conduct:

*I **Sudharsan D/192524063** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Sudharsan D/192524063

Annexure A

SCENARIO BASED TASK

QUESTIONS:

1 Scenario: Role of Parser vs Lexer

A student is developing a compiler for a simple calculator language where the lexical analyzer produces tokens such as NUM, PLUS, MINUS, STAR, DIVIDE, LPAREN, and RPAREN from the input expression. Although the lexer can identify and categorize symbols correctly, it does not ensure that the sequence of tokens forms a valid mathematical expression. The parser is responsible for analyzing this sequence based on grammar rules and constructing a syntax tree to represent the structure of the expression. In this scenario, the student must clearly differentiate between the roles of the lexer and parser and explain how the parser ensures syntactic correctness, handles operator precedence, and detects errors using a suitable example.

2 Scenario: Left Recursion Problem

A compiler developer is implementing a recursive descent parser and observes that the grammar $S \rightarrow Sa \mid b$ causes the parser to enter an infinite loop. This issue arises because the grammar contains immediate left recursion, which is not suitable for top-down parsing techniques. Since the parser repeatedly expands the same non-terminal without consuming any input, it leads to non-termination. In this scenario, the developer must identify the problem of left recursion and demonstrate how to systematically eliminate it by rewriting the grammar into a form that supports predictive parsing.

3 Scenario: Dangling Else Problem

A team is designing a predictive parser for the grammar $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$. While constructing the LL(1) parsing table, they encounter conflicts due to ambiguity in associating the else clause with the correct if statement. This situation is known as the dangling else problem. To analyze the issue, the team must compute the FIRST and FOLLOW sets of the non-terminals and explain how overlapping entries in the parsing table arise. The scenario requires identifying the source of ambiguity and explaining why the grammar is not suitable for LL(1) parsing.

4 Scenario: Shift-Reduce Parsing

A shift-reduce parser is used to process the input string $\text{id} + \text{id}$ using the grammar $E \rightarrow E + T \mid T$ and $T \rightarrow \text{id}$. The parser follows a bottom-up approach by maintaining a stack and performing shift and reduce operations to gradually derive the start symbol. At each step, the parser decides whether to shift the next input symbol onto the stack or reduce a sequence of symbols using a production rule. In this scenario, the parsing process must be traced step by step in a tabular format, showing the stack contents, remaining input, and the action taken at each stage.

5 Scenario: Operator Precedence Parsing

A parser encounters the expression $a / b * c$ and must evaluate it using operator precedence parsing techniques. The parser relies on precedence relations such as yields precedence, equal precedence, and takes precedence to determine whether to shift the next symbol or reduce the current handle. Since division and multiplication have the same precedence and are left associative, the parser must correctly apply these rules to ensure proper evaluation order. In this scenario, the precedence table must be constructed and the parsing process should be traced to show how the expression is evaluated.

6 Scenario: SLR Parsing Table Construction

A compiler student is assigned the task of constructing an SLR parsing table for the grammar $S \rightarrow AA$ and $A \rightarrow aA \mid b$. The process involves augmenting the grammar, generating the canonical collection of LR(0) items, computing FOLLOW sets, and building the parsing table with appropriate shift, reduce, and accept actions. Each state in the table represents a stage in the parsing process, and transitions between states are determined using grammar symbols. In this scenario, the student must describe the step-by-step procedure to construct the SLR parsing table and explain how it is used for efficient bottom-up parsing.

RUBRICS FOR CALCULATION:

Scenario based assessment

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. Scenario: Role of Parser vs Lexer:

The **lexical analyzer (lexer)** and **parser** perform different but complementary functions in a compiler.

The lexer operates at the **lexical level**. It reads the source program character by character and groups characters into meaningful tokens. For the calculator language, the lexer can generate tokens such as:

NUM, PLUS, MINUS, STAR, DIVIDE, LPAREN, RPAREN

For example, the expression:

$12 + 5 * 3$

may be converted into:

NUM PLUS NUM STAR NUM

The lexer only identifies the individual tokens. It does **not** determine whether those tokens form a syntactically valid mathematical expression.

The **parser** operates at the syntactic level. It receives the token sequence from the lexer and checks whether it follows the grammar of the language. It also constructs a **parse tree or syntax tree** representing the structure of the expression.

For example, using a grammar such as:

$$E \rightarrow E + T \mid E - T \mid T$$
$$T \rightarrow T * F \mid T / F \mid F$$
$$F \rightarrow \text{NUM} \mid (E)$$

the expression:

12 + 5 * 3

is interpreted as:

$$\begin{array}{c} + \\ /\backslash \\ 12 \quad * \\ \quad /\backslash \\ \quad 5 \quad 3 \end{array}$$

This demonstrates that * has higher precedence than +.

Error Detection:

Suppose the input is:

12 + * 5

The lexer can correctly identify:

NUM PLUS STAR NUM

However, the parser detects that + cannot be immediately followed by * according to the grammar. Therefore, the parser reports a **syntax error**.

Operator Precedence:

The parser uses grammar rules or precedence mechanisms to ensure that operators are evaluated correctly.

For:

12 + 5 * 3

the multiplication is performed before addition:

12 + (5 * 3)

rather than:

$(12 + 5) * 3$

Decision:

Therefore, the lexer should be responsible for **token recognition**, while the parser should handle **syntax validation, structural representation, precedence, associativity, and syntax-error detection**.

Conclusion:

The lexer answers “**What are the tokens?**”, whereas the parser answers “**Are these tokens arranged according to the grammar, and what is their structure?**” Both phases are necessary for reliable compiler construction. This directly addresses the scenario's distinction between lexical and syntactic analysis.

2. Scenario: Left Recursion Problem:

The given grammar is:

$$S \rightarrow Sa \mid b$$

This grammar contains **immediate left recursion** because the non-terminal **S** appears as the first symbol on the right-hand side of the production:

$$S \rightarrow S a$$

Why It Causes a Problem:

A recursive-descent parser is a **top-down parser**. If it tries to expand:

S

using:

$$S \rightarrow Sa$$

it again needs to expand **S** before consuming any input:

$$\begin{aligned} S &\Rightarrow Sa \\ &\Rightarrow Saa \\ &\Rightarrow Saaa \\ &\Rightarrow \dots \end{aligned}$$

The parser therefore enters an **infinite recursion** and never consumes the input.

Systematic Elimination:

The general form of immediate left recursion is:

$$A \rightarrow A\alpha \mid \beta$$

It can be converted into:

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' \mid \varepsilon$$

For the given grammar:

$$S \rightarrow Sa \mid b$$

we identify:

$$\alpha = a$$

$$\beta = b$$

Therefore, the left-recursion-free grammar becomes:

$$S \rightarrow bS'$$

$$S' \rightarrow aS' \mid \varepsilon$$

Verification:

The original grammar generates:

b

ba

baa

baaa

...

The new grammar generates the same language:

$$S \Rightarrow bS'$$

$$\Rightarrow baS'$$

$$\Rightarrow baaS'$$

$$\Rightarrow baaaS'$$

$$\Rightarrow baaa$$

where $S' \rightarrow \varepsilon$ terminates the derivation.

Decision:

The rewritten grammar is better for a recursive-descent parser because it removes immediate left recursion and allows the parser to make progress by consuming input.

Conclusion:

The original grammar is unsuitable for predictive top-down parsing because of infinite recursion. Converting it to:

$$S \rightarrow bS'$$
$$S' \rightarrow aS' \mid \varepsilon$$

eliminates the problem while preserving the language generated by the grammar. This is the appropriate solution for implementing a recursive-descent parser.

3. Scenario: Dangling Else Problem:

The grammar is:

$$S \rightarrow \text{if } E \text{ then } S$$
$$| \text{if } E \text{ then } S \text{ else } S$$
$$| a$$

This grammar suffers from the **dangling else ambiguity**.

Consider:

```
if E1 then
  if E2 then
    a
  else
    a
```

The **else** can potentially be associated with either the inner **if** or the outer **if**.

The conventional interpretation is that an **else** belongs to the **nearest unmatched if**.

FIRST Sets:

For the grammar:

$$S \rightarrow \text{if } E \text{ then } S$$
$$S \rightarrow \text{if } E \text{ then } S \text{ else } S$$
$$S \rightarrow a$$

we have:

$$\text{FIRST}(S) = \{ \text{if}, a \}$$

Both the first and second productions begin with **if**.

Therefore:

FIRST(if E then S)
=
FIRST(if E then S else S)
=
{ if }

FOLLOW Set:

Assuming S is the start symbol:

FOLLOW(S) = { \$, else }

The \$ represents the end of input, while else can follow an S in the second production.

LL(1) Conflict:

An LL(1) parser uses one lookahead token to select a production.

When the lookahead is:

if

both productions are possible:

$S \rightarrow \text{if } E \text{ then } S$
 $S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Thus, the same parsing-table entry would require two different productions.

This creates a **FIRST/FIRST conflict**.

Why It Is Not LL(1):

The parser cannot determine using only one lookahead symbol whether it should select:

$S \rightarrow \text{if } E \text{ then } S$

or:

$S \rightarrow \text{if } E \text{ then } S \text{ else } S$

Therefore, the grammar is ambiguous and is **not directly suitable for LL(1) parsing**.

Solution:

One common solution is to rewrite the grammar using matched and unmatched statements:

$S \rightarrow M \mid U$

$M \rightarrow \text{if } E \text{ then } M \text{ else } M$

| a

$U \rightarrow \text{if } E \text{ then } S$
| $\text{if } E \text{ then } M \text{ else } U$

This separates statements that have matched **if-else** pairs from statements containing an unmatched **if**.

Decision:

For a predictive parser, the grammar must be transformed or the parser must use a rule that associates **else** with the nearest unmatched **if**.

Conclusion:

The dangling else problem creates ambiguity because both productions beginning with **if** have the same FIRST set. Consequently, an LL(1) parsing table develops a conflict. Grammar transformation or a suitable parser rule is required to resolve the ambiguity.

4. Scenario: Shift-Reduce Parsing:

Given:

$E \rightarrow E + T \mid T$
 $T \rightarrow \text{id}$

Input:

id + id

Add the end marker:

id + id \$

A shift-reduce parser uses a **stack**, input buffer, and actions such as **Shift, Reduce, Accept**.

Parsing Table:

Step	Stack	Remaining Input	Action
1	\$	id + id \$	Shift id
2	\$ id	+ id \$	Reduce $T \rightarrow \text{id}$
3	\$ T	+ id \$	Reduce $E \rightarrow T$
4	\$ E	+ id \$	Shift +

5	\$ E + id \$	Shift id
6	\$ E + id \$	Reduce $T \rightarrow id$
7	\$ E + T \$	Reduce $E \rightarrow E + T$
8	\$ E \$	Accept

Step-by-Step Explanation:

Step 1:

The first input symbol is **id**, so it is shifted onto the stack.

Stack: \$ id

Step 2:

The stack contains **id**, which matches:

$T \rightarrow id$

Therefore, reduce **id** to **T**.

\$ T

Step 3:

Now **T** matches:

$E \rightarrow T$

So reduce:

\$ E

Step 4:

The next input is **+**. Shift it:

\$ E +

Step 5:

The next input is **id**, so shift it:

\$ E + id

Step 6:

Reduce:

$T \rightarrow id$

giving:

\$ E + T

Step 7:

The stack now matches:

E + T

Therefore:

$E \rightarrow E + T$

Reduce it to:

\$ E

Step 8:

The input contains only \$ and the stack contains the start symbol E.

Therefore:

Accept

Decision;

Shift-reduce parsing is appropriate here because it constructs the parse from the input symbols upward. It naturally supports **bottom-up parsing** and is suitable for grammars used by LR-family parsers.

Conclusion:

The input **id + id** is successfully recognized because the parser performs the correct sequence of shifts and reductions and finally reaches the start symbol E.

5. Scenario: Operator Precedence Parsing:

Given expression:

$a / b * c$

Division / and multiplication * have the **same precedence** and are **left associative**.

Therefore, the correct interpretation is:

$(a / b) * c$

and **not**:

$a / (b * c)$

Operator Precedence Relations:

Let:

id = operand

The important precedence relations can be represented as:

Stack Symbol	Input Symbol	Relation
\$	id	<
id	/	>
/	id	<
id	*	>
*	id	<
id	\$	>
/	*	<
*	\$	>

Here:

- < = yields precedence
- = = equal precedence
- > = takes precedence

Parsing Trace:

Input:

a / b * c \$

Step	Stack	Input	Action
1	\$	a / b * c \$	Shift a
2	\$ a	/ b * c \$	Reduce a
3	\$ E	/ b * c \$	Shift /
4	\$ E /	b * c \$	Shift b
5	\$ E / b	* c \$	Reduce b

6	\$ E / E	* c \$	Shift *
7	\$ E / E *	c \$	Shift c
8	\$ E / E * c	\$	Reduce c
9	\$ E / E * E	\$	Reduce E * E
10	\$ E / E	\$	Reduce E / E
11	\$ E	\$	Accept

Important Point:

Because / and * have equal precedence and are **left associative**, the parser must reduce the division before performing multiplication.

Thus:

$a / b * c$

is evaluated as:

$(a / b) * c$

Decision:

The precedence and associativity rules ensure that the parser does not incorrectly interpret the expression as:

$a / (b * c)$

Conclusion:

Operator precedence parsing provides a systematic method for deciding whether to shift or reduce. Since / and * have equal precedence and are left associative, the expression is correctly evaluated from left to right.